

HHBKTendo – Spielesammlung

HHBK Düsseldorf · Lernfeld 5 · 2026

- Strategiespiele mit MiniMax-KI
- Bauernschach & Tic-Tac-Toe (4 gewinnt)
- Tech-Stack: Python 3.10+ · SQLite · tkinter

Projektüberblick

■ Spielesammlung

- 2 Spiele auf je 6×6-Feld

■ KI-Gegner

- MiniMax + Alpha-Beta-Pruning
- Kein Code-Duplikat für beide Spiele

■ Benutzerverwaltung

- Login / Registrierung · SHA-256-Hashing · Gast-Modus

■ Bestenliste

- SQLite · Rangliste nach Schwierigkeitsgrad

Projektstruktur

- main.py → GUI, Screens, app_state
- minimax.py → generischer MiniMax + Alpha-Beta
- pawn_chess.py → Bauernschach Züge & Bewertung
- tictactoe.py → Tic-Tac-Toe Züge & Bewertung
- auth.py → Login & Registrierung (SHA-256 + Salt)
- database.py → SQLite CRUD (User & Ergebnisse)

- test_all.py → 15 Unittests (TC-01 bis TC-15)
- games.db → wird automatisch erstellt

Folie 4

Benutzerverwaltung & Datenbank

auth.py · database.py

auth.py – Kernfunktionen

- register() → Validierung + SHA-256 + Salt
- login() → Passwort-Hash-Vergleich
- save/load_session() → session.json für Auto-Login
- logout() → Session löschen
- is_guest() → Gast = keine Statistiken

SQLite-Tabellen

- users: id, username, password_hash, language, created_at
- results: id, user_id (FK), game, difficulty, won, played_at

Folie 5

DB-Techniken

row_factory · Schema-Migration · SQL-Injection-Schutz

- sqlite3.Row → Spaltenzugriff per Name: row['username']
- dict(row) → einfache Konvertierung für die App
- ?-Platzhalter → schützt vor SQL-Injection
- try/except bei ALTER TABLE → DB bleibt kompatibel

Folie 6

Sichere Passwortspeicherung

SHA-256 + Salt

- Kein Klartext – nur 'salt:hash' in der DB
- `os.urandom(16)` → 16 kryptografisch zufällige Bytes
- Salt macht Rainbow-Tables wirkungslos
- Gleiche Passwörter → verschiedene Hashes
- SHA-256 ist Einweg-Funktion

Folie 7

MiniMax-Algorithmus

mit Alpha-Beta-Pruning

- Rekursive Spielbaumsuche bis zur Tiefe N
- KI = Maximizer · Mensch = Minimizer
- Alpha-Beta: irrelevante Äste werden abgeschnitten

Schwierigkeitsgrade → Suchtiefe

- 1 Leicht = Tiefe 1 | 2 Mittel = Tiefe 2
- 3 Schwer = Tiefe 3 | 4 Experte = Tiefe 4
- 5 Meister = Tiefe 5 (sehr stark, mehrere Sekunden)

Folie 8

Callback-basiertes MiniMax-Design

- `get_best_move()` erhält Callbacks als Parameter:
 - `get_moves_fn` → welche Züge gibt es?
 - `apply_move_fn` → Zug ausführen
 - `evaluate_fn` → Bewertung des Zustands
- Kein Code-Duplikat: ein MiniMax für beide Spiele
- Single Responsibility: KI- & Spiellogik getrennt
- Einfach erweiterbar: drittes Spiel = neue Callbacks

Folie 9

Bauernschach

6×6 · Weiß (Mensch) vs. Schwarz (KI)

Spielfeld

- 6×6 Felder, nur Bauern · Weiß Reihe 6, Schwarz Reihe 1

Zugregeln

- Vorwärts 1 Feld (wenn frei)
- Diagonal schlagen möglich · Kein Doppelschritt

Gewinnbedingungen

- Gegnerische Grundlinie erreichen
- Alle Figuren des Gegners schlagen
- Gegner kann nicht mehr ziehen

Bewertungsfunktion

- Figurenzahl + Fortschritt + Zentrumskontrolle

Folie 10

Bewertungsfunktion Bauernschach

Heuristik: Figurenzahl · Fortschritt · Zentrum

- Figurenzahl: +10 pro eigener Bauer, -10 pro Gegner
- Fortschritt: +row×2 → KI wird für Vorwärtsbewegung belohnt
- Zentrum: +0 bis +3 (Zentralposition strategisch wertvoller)
- Positiv = gut für KI (Schwarz) · Negativ = gut für Mensch
- CENTER_BONUS-Matrix definiert Feldwertigkeit

Folie 11

Tic-Tac-Toe (4 gewinnt)

6×6 · Mensch (X) vs. KI (O)

Spielregeln

- Abwechselnd Stein setzen · Mensch (X) beginnt
- 4 in einer Reihe gewinnt (H/V/Diagonal)
- Unentschieden bei vollem Feld

KI-Optimierungen

- Kandidatenzüge: nur Nachbarmfelder (36 → 8–15 Züge)
- Center-Sorting: Zentrumsnähe für Alpha-Beta

Bewertungsfunktion

- Dreier-Reihe: +50 / -50 · Zweier-Reihe: +10 / -10
- Mittelfeld-Bonus

Folie 12

TTT Optimierungen

Kandidatenzüge · Center-Sorting · shallow copy

- Nur Nachbarmfelder besetzter Felder als Kandidaten
- Center-Sorting → Alpha-Beta findet früh gute Äste
- `[row[:]]` for `row` in `board`] statt `deepcopy` → ~5× schneller

Folie 13

GUI & Threading

daemon Thread · `after(0, ...)` · `app_state`

- `app_state{}` als globaler Zustand (prozedural, kein OOP)
 - `board`, `ai_thinking`, `difficulty`, `current_user`, ...
- KI-Zug in separatem Thread → GUI bleibt reaktionsfähig
- `daemon=True` → Thread endet mit Fenster
- `after(0, fn)` → UI-Updates sicher in tkinter-Hauptthread
- tkinter ist nicht thread-safe → NIE Widgets aus Thread ändern

Folie 14

Unittests

TC-01 bis TC-15 · test_all.py · pytest-kompatibel

- TC-01–03: Passwort-Hashing (Generierung, Verifikation, Salt)
- TC-04–06: Benutzer-DB (Registrierung, Duplikat, Abfrage)
- TC-07–08: Spielergebnisse (Speichern, Bestenliste)
- TC-09–10: Bauernschach (Züge, Anwenden, Gewinner)
- TC-11–12: Tic-Tac-Toe (4-in-Reihe, Kandidatenzüge)
- TC-13–15: MiniMax (offens. Zug, Gewinn erkennen, Blockieren)

Folie 15

Test-Infrastruktur

unittest · Temp-DB-Isolation · Mocking

- DBTestCase: jeder Test bekommt frische Temp-DB
- tempfile.mkstemp('.db') → isolierte Datenbankdatei
- patch.object(database, 'DB_PATH') → keine Produktions-DB
- assertEquals testet 4-in-Reihe-Erkennung direkt

Folie 16

Zusammenfassung

- ■ Zwei vollständige Strategiespiele auf 6×6-Feld
- ■ Wiederverwendbarer MiniMax + Alpha-Beta-Pruning
- ■ Sichere Authentifizierung: SHA-256 + Salt
- ■ SQLite: keine externen Abhängigkeiten
- ■ tkinter-GUI: Windows / macOS / Linux
- ■ 15 Unittests sichern Kernfunktionen ab

Folie 17

SOLL/IST Feature-Vergleich

Lastenheft → Pflichtenheft → Umsetzung → Test

MUSS-Ziele: 12/12 ■

- 2 Spiele · 6×6 Brett · 5 Schwierigkeitsgrade
- Login · Registrierung · Gastmodus · Spielabbruch · Regelanzeige
- Bestenliste · SHA-256-Hashing · MiniMax-KI · Unittests

SOLL-Ziele: 2/2 ■

- Alpha-Beta-Pruning · SQL-Bestenliste

KANN-Ziele: 2/4 ■

- ■ Sprachwechsel EN/DE · Sprachpräferenz in DB
- ■ Dame (drittes Spiel) · Alt-KI-Schnittstelle (zeitbedingt)

Folie 18

SOLL/IST Projektplanung

Wasserfallmodell · 2 Meetings · Discord & WhatsApp

- Analysephase (Woche 1: 20.04.–24.04.) ■ plangemäß
- Implementierung (27.04.–04.05.) ■ +1 Tag (01.05. Feiertag)
- Test & Doku: parallel zur Impl. statt sequenziell ■
- Abschluss (Woche 3) ■ plangemäß
- Discord-Channel statt Kanban-Board
- 2 Team-Meetings: 27.04. & 29.04. je 10:00 Uhr

Folie 19

Benutzeroberfläche (GUI)

tkinter · HHBKTendo Design System · Dark Theme

Screen-Flow

- ① Login → Login / Register / Als Gast spielen

- ② Hauptmenü → Spiel · Schwierigkeit · Play · Bestenliste
- ③ Spielscreen → 6×6 Brett · Züge per Klick

Design-System (Farben)

- #0D0D1A Deep Navy (BG) · #E94560 Hot Pink (Primär)
- #6C3FC5 Royal Purple · #00D4FF Cyber Cyan (Akzent)

tkinter-Besonderheiten

- Buttons als tk.Label + Bindings (macOS-Bug: bg/fg)
- KI in separatem Thread · after(0, cb) für GUI-Rückkehr

Folie 20

Quellen & Werkzeuge

Entwicklungsumgebungen

- PyCharm (JetBrains) · Visual Studio Code (Microsoft)

KI-Assistenten

- Claude (Anthropic) · ChatGPT (OpenAI)

Python-Bibliotheken (alle stdlib außer Pillow)

- tkinter · sqlite3 · hashlib · threading · Pillow (GIF)

Quellen

- Wikipedia: MiniMax · Alpha-Beta-Pruning
- HHBK Lernfeld 5 – Unterrichtsmaterial
- docs.python.org – Python-Dokumentation

Folie 21

Fazit – Projektverlauf

Team & Rollen

- Lucas – Projektleitung, auth.py, Testing
- Jesse – database.py, Pitch, Testing
- Arda – database.py, Testing

- Uilliam – GUI (main.py), Design
- Jakub – pawn_chess.py · Niklas – minimax.py

Highlights

- Spielunabhängige MiniMax-Schnittstelle
- Reibungslose Teamarbeit dank klarem Pflichtenheft

Lessons Learnt

- Kanban-Board früher einsetzen
- Schnittstellen detaillierter im Pflichtenheft definieren

Ausblick

- Dame als drittes Spiel · Sprach-Auswahl ausbauen